2.1 Algorithms

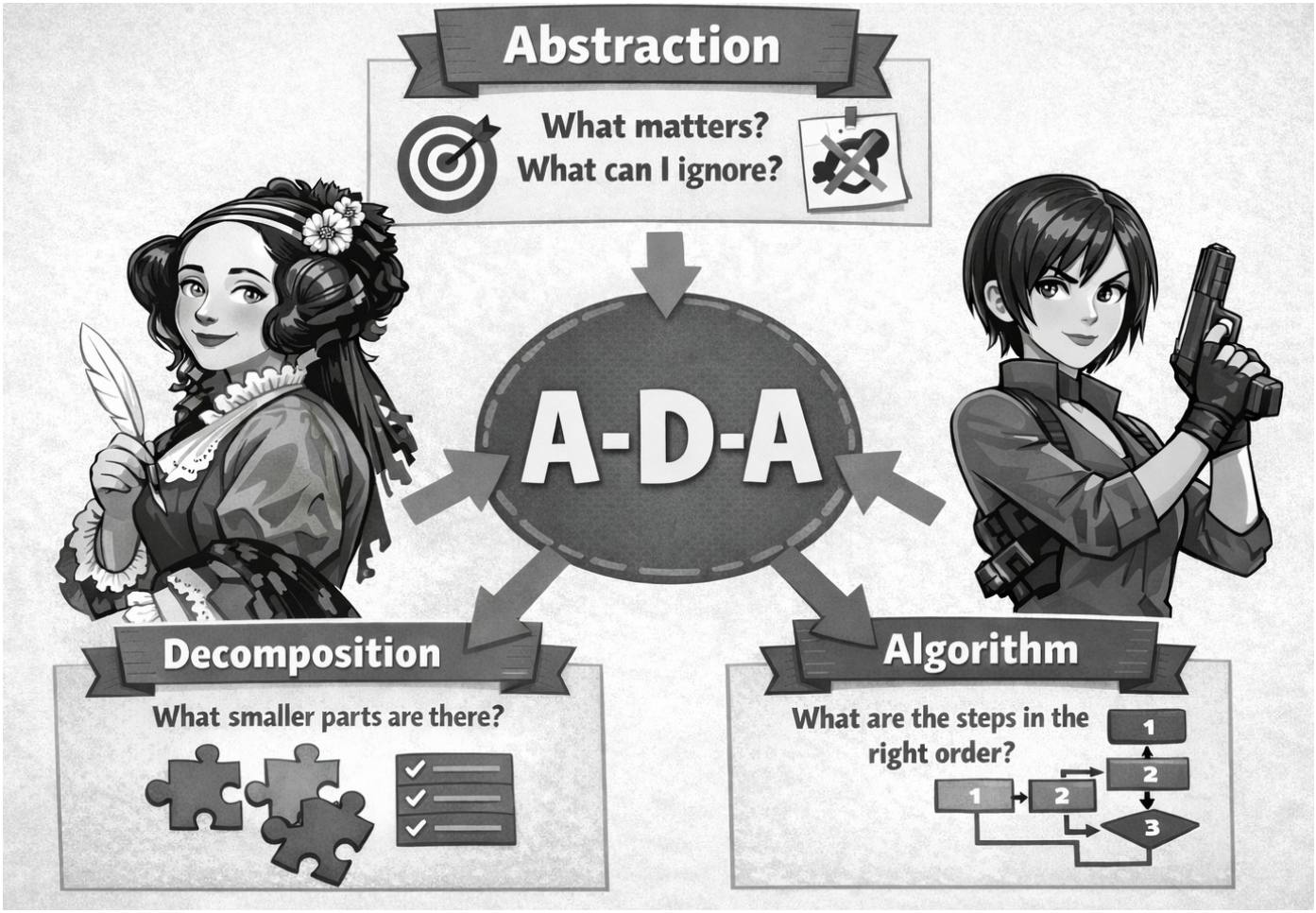
Keyword	Definition
Computational Thinking	The process of thinking about a problem using computational means in order to create a solution which a computer can implement.
Abstraction	The process of removing irrelevant or unnecessary information from the problem in order to better understand the basic parts of it.
Decomposition	The process of breaking a problem down into smaller parts to make it easier to solve.
Algorithmic Thinking	Designing ordered, logical steps to solve a problem and create a solution.
Algorithms	A sequence of logical instructions to carry out a particular task.
Structure Diagram	A diagram which shows in graphical format the steps needed to solve a problem. There are several different types of structure diagram, and a flow chart is one example.
Pseudocode	A way of writing out algorithms or other computer programs in a way which sits somewhere between full sentences and computer code.
Flowchart	A way of using pictures and shapes to represent the steps needed to solve a problem or that a program will take.
Programming Language	A language which can be understood by computers to produce computer software.
Trace Table	A table which records the values stored in each of a program's variables at each stage through the program. It is a useful tool when trying to find errors in the code.
Search	Finding a value or piece of information within a set of data.
Binary Search	A way of searching an ordered list by looking at the middle item in the list, then looking at either the items above or below depending on if the required value is higher or lower. This process repeats until the value is found.
Linear Search	A way of searching through a list items, checking them one at a time until the required value is found.
Sort	A way of placing information into a certain order, such as alphabetically or by number.
Bubble Sort	A way or sorting a list by comparing the items in a list, two at a time, and changing the order if needed. The process repeats until the list is in the correct order.
Merge Sort	A way of sorting a list by dividing it in two until each item is separated individually. Pairs of items are compared and placed in order until the list as a whole is ordered.
Insertion Sort	A way of sorting a list by working through the items one at a time and placing them in the correct position in the list.

2.1.1 Computational Thinking

2.1.1 Computational thinking	
☐ Principles of computational thinking: ○ Abstraction ○ Decomposition ○ Algorithmic thinking	**Required** ✓ Understanding of these principles and how they are used to define and refine problems

Computational thinking is a way of solving problems by breaking them into smaller parts, ignoring unimportant details, and writing clear step-by-step instructions, for example, planning a GCSE revision timetable: list subjects → split each into topics → pick the most important areas → plan short revision sessions → test yourself and adjust each week.

Principle	What it means (simple)	Why it helps	Quick example
Abstraction	Keep the **important** details, ignore what you **don't need**	Makes the problem simpler and clearer	Bus app: keep stop names/times, ignore seat colour
Decomposition	Break a big problem into **smaller tasks**	Easier to solve, build, and test	Game: movement, scoring, collisions, menu
Algorithmic thinking	Plan **step-by-step instructions** in the right order	A computer can follow the steps exactly	Login: input user/pass → check → allow/deny



Quick exam sentence starters

- **Abstraction:** "I will ignore unnecessary details like… and keep…"
- **Decomposition:** "I will split the problem into…"
- **Algorithmic thinking:** "I will create steps in order: first…, then…, next…

2.1.1 Exam questions on Computational Thinking

Question	Tip to answer	Common misconception
1. Which option is the best example of **decomposition**? A) Ignoring the colour of a phone case B) Splitting a game into levels, scoring and controls C) Writing code in Python D)Using Wi-Fi to connect devices **(1)** B	**Tip:** Decomposition = split it up.	✘ Picking abstraction (ignoring detail).
2. Which statement describes **abstraction**? A) Breaking a problem into smaller parts B) Testing a program C) Keeping important details and ignoring the rest D) Writing steps in order **(1)** C	**Tip:** Abstraction = ignore the fluff.	✘ Mixing it up with decomposition.
3. What is meant by **computational thinking**? **(1)** Computational thinking is a way to solve problems by thinking in clear, logical steps.	**Tip:** Use "solve problems" + "logical steps".	✘ Talking only about coding.
4. What is an **algorithm**? **(1)** An algorithm is a set of step-by-step instructions to solve a problem.	**Tip:** Say step-by-step instructions.	✘ Calling it "code".
5. State **two** computational thinking techniques. **(2)** Abstraction.$_1$ Decomposition.$_2$	**Tip:** Two techniques = two marks.	✘ Giving examples instead of naming them.
6. State **two** features of a problem that make it **computable**. **(2)** You can write clear step-by-step instructions (an algorithm).$_1$ The steps are precise and repeatable.$_2$	**Tip:** Think: can I write exact steps?	✘ Saying "needs a computer".
7. A school is making a **lunch pre-order app**. Explain how **abstraction** helps when planning the app. **(3)**	**Tip:** Say what you keep + what you	✘ Listing features with no explanation.

Abstraction keeps only important information.[1] Keep meal choice/allergies/payment.[2] Ignore details that don't affect ordering.[3]

---

8. A student is designing a **multiplayer racing game**. Explain how **decomposition**helps. **(3)**

Split into parts like controls, lap counting and leaderboard.[1] Build each part separately.[2] Then combine into one full game.[3]

Tip: Name 3 mini-tasks.

✘ Only saying "it makes it easier".

---

9. Explain why **not all problems** can be solved by a computer. **(3)**

A computer needs an algorithm (clear steps).[1] Some problems can't be written as exact steps.[2] So a computer can't reliably solve them.[3]

Tip: Use the word algorithm.

✘ "Computers aren't smart" (too vague).

---

10. Match each technique to the best example. Techniques: Abstraction / Decomposition / Algorithmic thinking.

Examples:
A) Plan the exact order of steps for a password reset
B) Split a social media app into feed, messages and settings
C) Ignore profile picture style and keep username + age only

(3)

Algorithmic thinking → A.[1] Decomposition → B.[2] Abstraction → C.[3]

Tip: One technique per example.

✘ Linking more than once.

---

11. Compare **abstraction** and **decomposition**. **(4)**

Both make problems easier.[1] Abstraction removes unneeded detail.[2] Decomposition splits into smaller tasks.[3] Example given in context.[4]

Tip: Same + different + example.

✘ Only describing one technique.

---

12. A college wants a system that tracks **attendance** and messages home if it drops. Explain how computational thinking techniques help you **define and refine** the problem. **(6)**

Decompose into register/store/calculate/message.[1] Abstract to keep only needed data.[2] Set clear rules (e.g. threshold).[3] Write ordered steps for the process.[4] Makes testing easier.[5] Final solution is clearer and more reliable.[6]

Tip: 3 short chunks: Decomposition, Abstraction, Algorithmic thinking.

✘ Forgetting to refine rules/inputs/outputs.

02 KEY VOCABULARY / 2.1.1

Computational thinking

Use the correct technical wording before attempting longer answers.

Decomposition

Breaking a large problem into smaller tasks that can each be solved.

Abstraction

Identifying the essential information and removing irrelevant detail.

Algorithmic thinking

Planning a logical order of unambiguous steps that solves a problem.

Inputs and outputs

Data a solution receives and the useful result it produces.

03 WORKED EXAMPLE / 2.1.1

Computational thinking

Follow a complete example. Notice the method before attempting the separate questions.

Task
Design a simple lunchtime pre-order system.

Think it through — small steps

1. Decompose the system into collecting an order, calculating the cost and displaying confirmation.

2. Abstract away irrelevant details such as the colour of the queue barrier.

3. Specify necessary data: customer ID, food item, quantity and price.

4. Order the solution: input order, calculate amount, check availability, display result.

Why this earns marks
Each method is named, applied to the problem and shown in a clear sequence.

SPEC 2.1.1 | SKILL 01 OF 28

Abstraction and decomposition

REMEMBER

Decomposition breaks a large problem into manageable parts. Abstraction keeps relevant details and ignores irrelevant ones.

THINK OF IT LIKE...

Plan a school trip: separate travel, permissions and costs; ignore the colour of the coach seats.

WORKED EXAMPLE / APPLY

For a bus timetable app, store stops and arrival times, not passengers' favourite foods.

YOU DO - EXAM PRACTICE

Explain the difference between decomposition and abstraction. (2)

Write your answer before opening the separate answer section.

SPEC 2.1.1 | SKILL 02 OF 28

Algorithmic thinking

REMEMBER

Algorithmic thinking produces a logical, ordered sequence of steps that could solve a problem.

THINK OF IT LIKE...

Like a recipe: the order of instructions matters.

WORKED EXAMPLE / APPLY

To find the larger of two numbers: input both, compare them, output the larger.

YOU DO - EXAM PRACTICE

Describe algorithmic thinking and give one benefit. (2)

Write your answer before opening the separate answer section.

04 INDEPENDENT EXAM PRACTICE / 2.1.1

Computational thinking

Do this without viewing the next answer pages. Use a whiteboard first for code/traces.

EXAM-STYLE CHALLENGE · 5 MARKS

A school wants an app for booking school lunches. Explain how computational thinking could help the developers plan this app. Include decomposition, abstraction and a logical algorithm.

Your response

PREDICT → WRITE → TEST → IMPROVE | Self-mark after finishing the entire topic.

MARK SCHEME | SKILL 01 OF 28

Abstraction and decomposition

QUESTION

Explain the difference between decomposition and abstraction. (2)

MODEL ANSWER

Decomposition separates a complex task into smaller tasks (1). Abstraction removes irrelevant information to focus on what matters (1).

SELF-MARK

Give credit for equivalent correct wording. Correct your answer in a different colour, then revisit the study card if a key point was missing.

MARK SCHEME | SKILL 02 OF 28

Algorithmic thinking

QUESTION

Describe algorithmic thinking and give one benefit. (2)

MODEL ANSWER

It involves designing a logical step-by-step method to solve a problem (1). Precise steps can be followed, tested and implemented consistently (1).

SELF-MARK

Give credit for equivalent correct wording. Correct your answer in a different colour, then revisit the study card if a key point was missing.

05 ANSWERS & MARK SCHEME / 2.1.1

Computational thinking

This answer belongs to the exam-style challenge on the previous practice page.

5-mark challenge · Model answer
Lunchtime ordering system

Mark 1 Decomposition: identify a manageable subtask, such as choosing meals or checking stock.

Mark 2 A second distinct subtask, such as taking payment or producing confirmation.

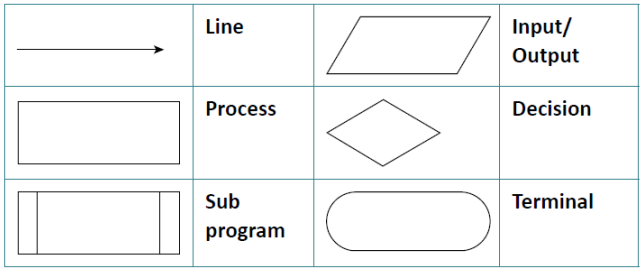
Mark 3 Abstraction: identify relevant data, e.g. meal choice, allergy and collection time.

Mark 4 Abstraction: reject irrelevant detail, e.g. preferred colour of the dining table.

Mark 5 Algorithmic thinking: give ordered steps from receiving a choice to checking stock and producing a confirmation.

Accept equivalent correct methods, code or relevant wording. Award each point once.

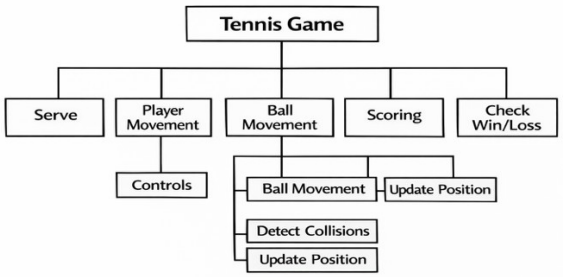
2.1.2 Designing, Creating and Refining Algorithms

2.1.2 Designing, creating and refining algorithms	
☐ Identify the inputs, processes, and outputs for a problem ☐ Structure diagrams ☐ Create, interpret, correct, complete, and refine algorithms using: ○ Pseudocode ○ Flowcharts ○ Reference language/high-level programming language ☐ Identify common errors ☐ Trace tables	**Required** ✓ Produce simple diagrams to show: ▪ The structure of a problem ▪ Subsections and their links to other subsections ✓ Complete, write or refine an algorithm using the techniques listed ✓ Identify syntax/logic errors in code and suggest fixes ✓ Create and use trace tables to follow an algorithm ✓ Use of nesting for selection and iteration **Flowchart symbols** 

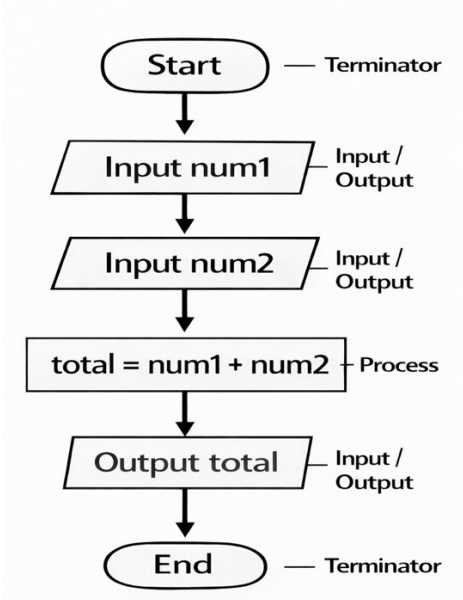
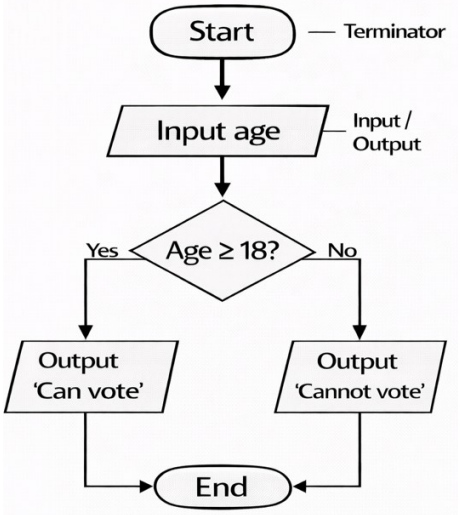
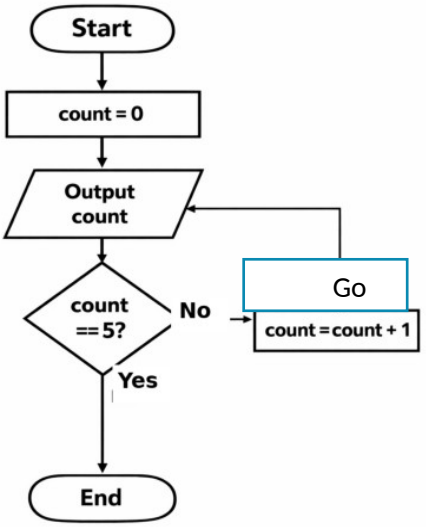
An algorithm is set of instructions used to solve a problem. It is **designed** by identifying the inputs, the processes needed to manipulate the data, and the outputs produced by the program.

Stage	What it means (simple)	Common examples	Exam tip
Input	Data that goes **into** the program	User enters a number Username / password / Data read from a file	Look for `input()` or file reading
Process	What the program **does** with the data	Calculations IF statements Loops (FOR / WHILE) Searching or sorting	Use selection and iteration
Output	The result the program **produces**	Message on screen Total or result Data written to a file	Usually uses `print()`

Structure Diagram

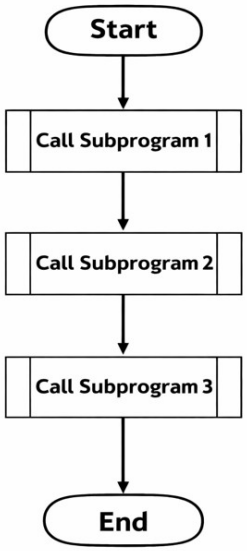


Reason	Why it matters
Planning	Makes complex problems easier
Decomposition	Breaks big problems into parts
Coding	Each section can become a procedure or function
Debugging	Errors are easier to find
Exams!	Often linked to 3–6 mark questions

Scenario	Flowchart	Pseudocode
Sequence: ***Add two numbers and output the total***		num1 = input("Enter first number") num2 = input("Enter second number") num3 = input("Enter third number") total = num1 + num2 + num3 print(total) (*Exam note - no datatypes in Pseudocode)*
Selection ***Check if someone is old enough to vote (18+)*** Decision (Yes or No)		age = input("Enter age") **if** age >= 18 **then** print("Can vote") **else** print("Too young") endif *(Exam note - no datatypes in Pseudocode)*
Iteration ***Count from 0 to 5 and output each number*** Go Back - That's iteration !		for count =0 to 5 print(count) next count

Subprograms

Subprograms are named blocks of code that do one specific job, and only run when they are **called**.



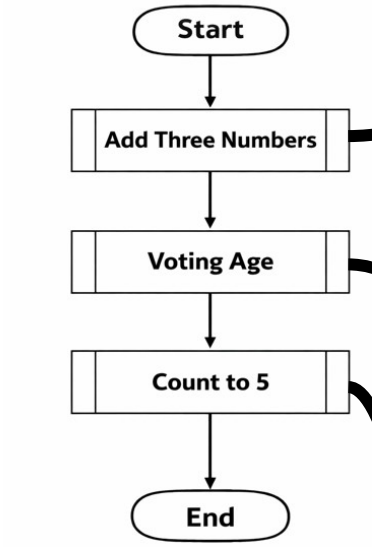
Why use subprograms?
Make programs **shorter and easier to read**
Avoid **repeating** the same code
Make testing and fixing bugs **easier** (you can test one part at a time)

Two types you need to know

Procedure
Runs code but **does not return** a value
Used for tasks like displaying a menu, printing a message, saving to a file

Function
Runs code and **returns** a value back to the main program
Used for calculations like total price, average score, converting units.

More of this in Section 2.2!

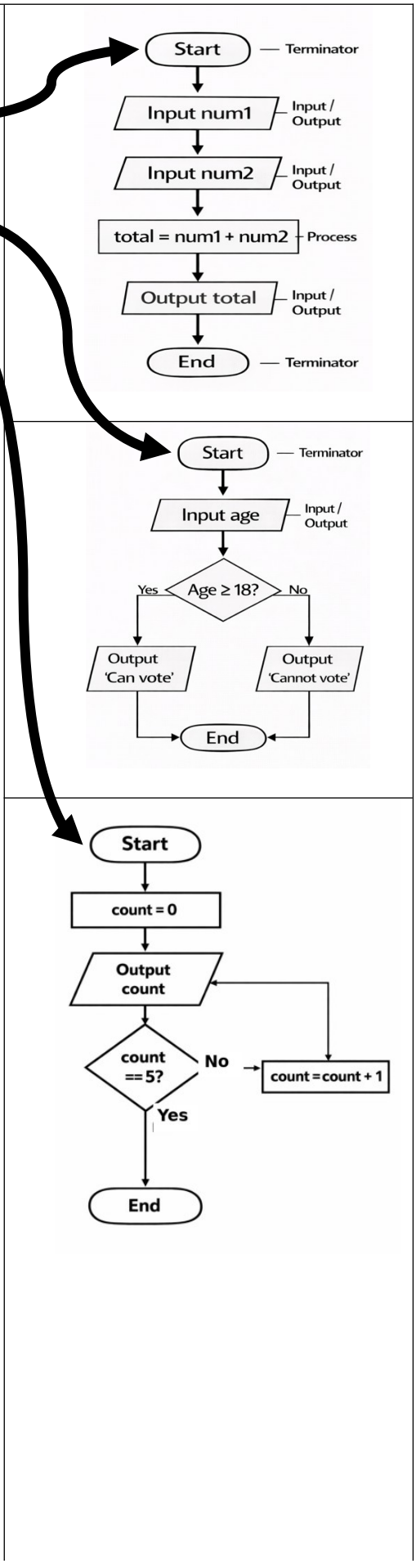



```
Procedure
AddThreeNumbers()
    num1 = input("Enter first
        number")
    num2 = input("Enter second
        number")
    num3 = input("Enter third
        number")
    total = num1 +  num2 +
        num3
    print(total)
Endprocedure

Procedure VotingAge()
    age = input("Enter age")
    if age >= 18 then
        print("Can vote")
    else
        print("Too young")
    endif
Endprocedure

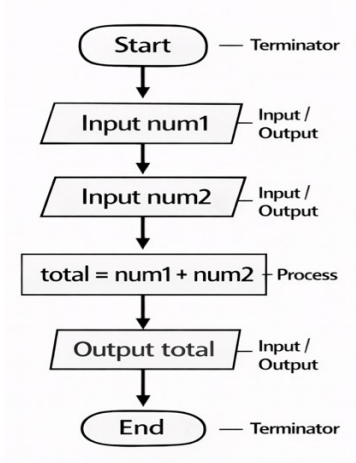
Procedure CountTo5()
    for count =0 to 5
        print(count)
    next count
Endprocedure

//Main
    AddThreeNumbers()
    VotingAge()
    CountTo5()
```



A **trace table** is a simple table you fill in to **track what happens to variables step by step** as an algorithm runs.Used to **check your logic works** (spot mistakes early)

Flowchart



Pseudocode

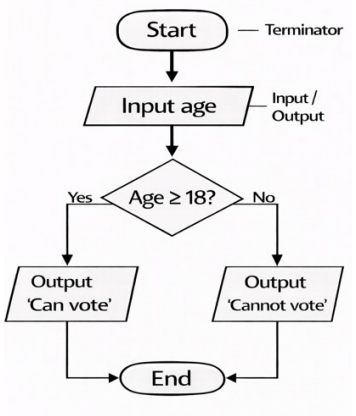
Line	Code
1	num1 = input("Enter first number")
2	input("Enter second number")
3	total = num1 + num2
4	print(total)

Trace Table

User enters 6 then 7

Line	num1	num2	total	Output
1	6			
2	6	7		
3	6	7	13	
4	6	7	13	13

Flowchart



Pseudocode

Line	Code
1	age = input("Enter age")
2	If age>=18 then
3	print("Can vote")
4	**else**
5	print("Too young")
6	endif

Trace Table

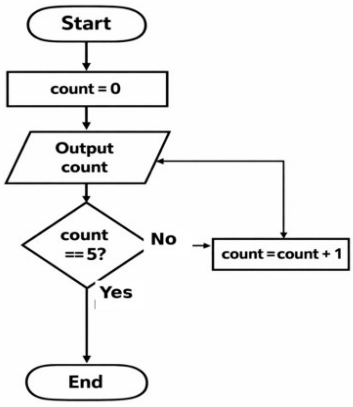
User enters 3

Line	age	Output
1	3	
2		
5		"Too young")

User enters 18

Line	age	Output
1	18	
2		
3		"Can Vote")

Flowchart



Pseudocode

Line	Code
1	for count =0 to 5
2	print(count)
3	next count

Trace Table

Line	count	Output
1	1	
2		1
3	2	
2		2
3	3	
2		3
3	4	
2		4
3	5	
2		5
3	6	

Trace an iteration – worked method: For x = 8, the statements print("GO"); x = x - 3 repeat while x > 0. Track the value before each output: 8, 5, 2; the message appears three times. Always check whether the condition is tested before or after the loop body.

2.1.2 Exam questions on Designing, Creating and Refining Algorithms

Question	Tip to answer (student)
1. Which method is BEST for writing an algorithm that is easy for any programmer to understand? (Tick one) (1) ☐ Everyday English ☐ Pseudocode ☐ A specific programming language ☐ Machine code Pseudocode (1)	Think: program language independent.
2. A programmer writes an algorithm in everyday English. Explain one problem with this. (2) Different people may understand the steps differently.[1] This can lead to different outcomes or mistakes.[2]	"Open to interpretation" = key phrase.
3. State **two** reasons why writing an algorithm in a specific programming language can be a bad idea at the design stage. (2) It is tied to one language (not portable).[1] It can be slower to write than pseudocode/flowcharts.[2]	Two separate reasons.
4. Name **two** ways algorithms are commonly represented (2) Flowcharts[1] and pseudocode.[2]	Just name both.
5. Complete the missing word: "Pseudocode is _____ dependent and uses no fixed syntax." (1) language [1]	OCR phrase.
6. Identify the correct flowchart symbol for a decision. (Tick one) ☐ Rectangle ☐ Parallelogram ☐ Diamond ☐ Oval (1) Diamond [1]	Decision = question/IF.
7. A flowchart contains an input box and an output box. Describe what each one does. (4) Input box: gets data from the user/sensor/file.[1] It stores the value in a variable for the program to use.[2] Output box: shows results to the user (screen/sound/print).[3] It displays the value/message produced by the algorithm.[4]	2 marks each idea: input vs output.
8. A student is asked to "correct an algorithm". What does this usually mean? (2) Find an error in the steps/logic.[1] Fix it so the algorithm works	Mention logic / missing steps.

Question	Tip to answer (student)
9. A program should only accept a username that is **6 to 12 characters** long. Complete the condition: IF length(username) ____ 6 AND length(username) ____ 12 THEN (2) >= 6[1] <= 12[2]	Use >= and <= (inclusive).
10. A programmer makes this mistake: IF score > 10 AND < 20 THEN Explain why this is incorrect and give the correct version. (3) The second part is not a full comparison (it does not include the variable).[1] You must compare score in both conditions.[2] Correct: IF score > 10 AND score < 20 THEN.[3]	Must compare the variable twice.
11. Describe how a **trace table** helps when refining an algorithm. (3) You follow the algorithm step by step.[1] You record how variables change after each step.[2] This helps spot where the logic goes wrong so it can be fixed.[3]	Mention values changing + finding bugs.
12. A dry run is carried out on an algorithm. What is a dry run? (2) Testing the algorithm without coding it.[1] You step through the logic using sample inputs to check outputs.[2]	"Run it in your head".
13. A flowchart is incomplete. State **two** things you should check before adding the missing symbols. (2) Follow the arrows to understand the current logic.[1] Check the meaning of symbols so you add the correct type (process/decision/I/O).[2]	Two checks.
14. A programmer is designing an algorithm for a school canteen system. They start coding immediately and do not plan the algorithm first. Explain **two** problems this could cause. (4) They may misunderstand the problem and miss key requirements.[1] This leads to a solution that does not match what is needed.[2] They are more likely to make logic errors because steps are not planned.[3] Fixing mistakes later can take longer than planning first.[4]	Think: bugs + missed requirements + messy code.

02 KEY VOCABULARY / 2.1.2

Designing and tracing algorithms

Use the correct technical wording before attempting longer answers.

Flowchart

A diagram using standard symbols and arrows to show an algorithm.

Pseudocode

Structured text that represents an algorithm without requiring a specific language.

Trace table

A record of variable values as statements execute, used to check logic.

Selection

Choose between paths using a condition.

Iteration

Repeat instructions while a condition holds or for a set count.

03 WORKED EXAMPLE / 2.1.2

Designing and tracing algorithms

Follow a complete example. Notice the method before attempting the separate questions.

Task
Trace a loop that accumulates numbers.

Think it through — small steps

1. Start with total = 0.
2. Add 2, then 4, then 6 to the running total.
3. Record total after each step: 2, then 6, then 12.
4. Check the loop has stopped; output the final total.

Code / trace example

```
total = 0
for n in [2, 4, 6]:
    total = total + n
print(total)  # 12
```

Why this earns marks
A correct trace shows the changing value at every iteration, not just the final number.

SPEC 2.1.2 | SKILL 03 OF 28

Pseudocode and flowcharts

REMEMBER

Pseudocode describes an algorithm in structured text. A flowchart uses symbols and arrows to show control flow.

THINK OF IT LIKE...

Both are route maps: one written, one drawn.

WORKED EXAMPLE / APPLY

INPUT age; IF age >= 16 THEN OUTPUT "Eligible" ELSE OUTPUT "Not eligible" ENDIF.

YOU DO - EXAM PRACTICE

Identify the flowchart symbol used for a decision and explain its purpose. (2)

Write your answer before opening the separate answer section.

SPEC 2.1.2 | SKILL 04 OF 28

Trace tables

REMEMBER

A trace table records the value of variables each time important statements execute.

THINK OF IT LIKE...

A journey log for a running algorithm.

WORKED EXAMPLE / APPLY

total = 0; FOR i = 1 TO 3: total = total + i. Values of total: 1, 3, 6.

YOU DO - EXAM PRACTICE

A loop adds 2, then 4, then 6 to total = 0. Give total after each addition. (3)

Write your answer before opening the separate answer section.

04 INDEPENDENT EXAM PRACTICE / 2.1.2

Designing and tracing algorithms

Do this without viewing the next answer pages. Use a whiteboard first for code/traces.

EXAM-STYLE CHALLENGE · 6 MARKS

Trace the algorithm shown below. Give count and total AFTER each iteration (four rows), explain why the loop stops, and give the printed value.

```
count = 1
total = 0
while count <= 4:
    total = total + count
    count = count + 1
print(total)
```

Your response

PREDICT → WRITE → TEST → IMPROVE | Self-mark after finishing the entire topic.

MARK SCHEME | SKILL 03 OF 28

Pseudocode and flowcharts

QUESTION

Identify the flowchart symbol used for a decision and explain its purpose. (2)

MODEL ANSWER

A diamond represents a decision (1). It tests a condition and branches according to its result, normally Yes/No (1).

SELF-MARK

Give credit for equivalent correct wording. Correct your answer in a different colour, then revisit the study card if a key point was missing.

MARK SCHEME | SKILL 04 OF 28

Trace tables

QUESTION

A loop adds 2, then 4, then 6 to total = 0. Give total after each addition. (3)

MODEL ANSWER

After 2: 2 (1). After 4: 6 (1). After 6: 12 (1).

SELF-MARK

Give credit for equivalent correct wording. Correct your answer in a different colour, then revisit the study card if a key point was missing.

05 ANSWERS & MARK SCHEME / 2.1.2

Designing and tracing algorithms

This answer belongs to the exam-style challenge on the previous practice page.

6-mark challenge · Model answer
Trace a four-iteration loop

Mark 1 First row: count=2, total=1.

Mark 2 Second row: count=3, total=3.

Mark 3 Third row: count=4, total=6.

Mark 4 Fourth row: count=5, total=10.

Mark 5 The loop stops because 5 is not <= 4.

Mark 6 The value printed is 10.

Accept equivalent correct methods, code or relevant wording. Award each point once.

WORKED SOLUTION / 2.1.2

Designing and tracing algorithms

See a complete method and compare it with each mark-scheme point.

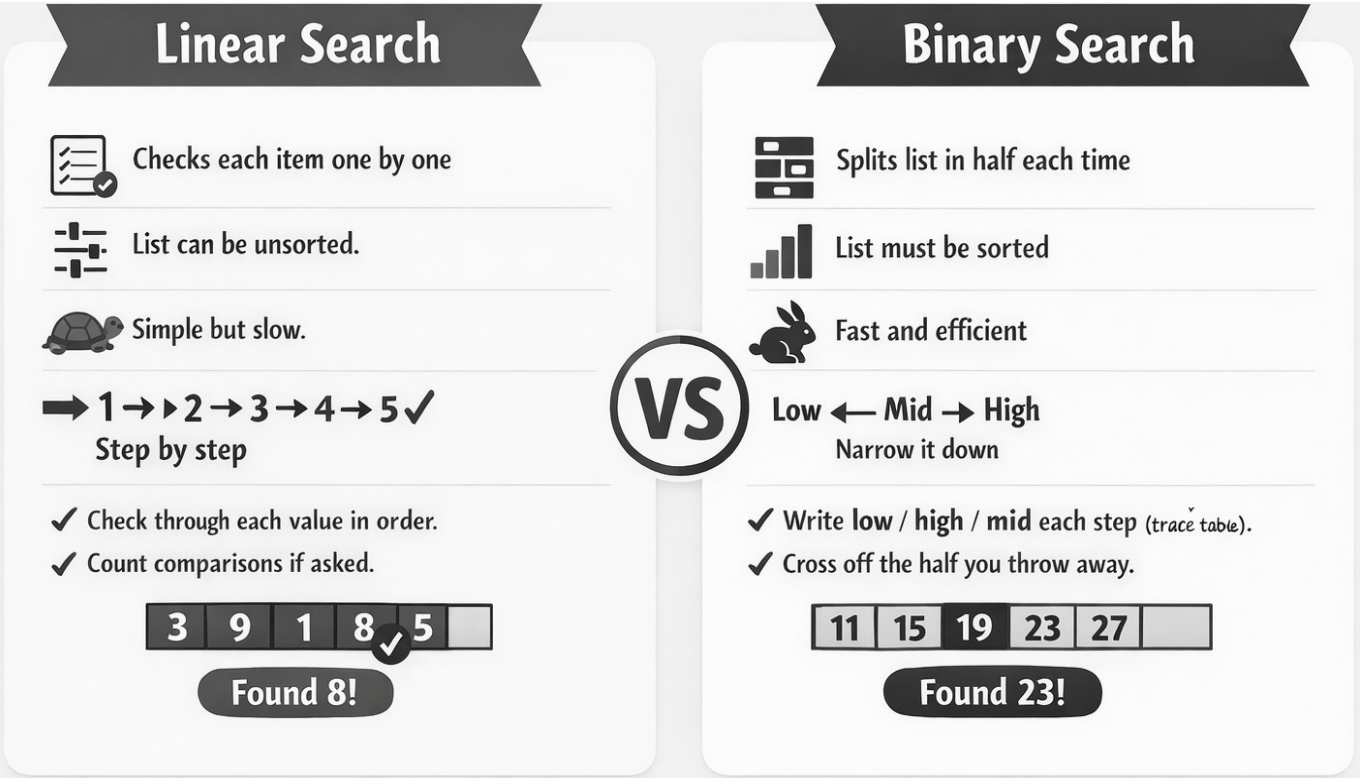
Algorithm
count starts at 1; total starts at 0. While count <= 4, add count to total and then increase count by 1.

Iteration	count AFTER	total AFTER
1	2	1
2	3	3
3	4	6
4	5	10

Result
The loop stops when count becomes 5, because 5 <= 4 is false. The final output is **10**. Record the variables after each update, not just the last result.

2.1.3 Searching and Sorting Algorithms

2.1.3 Searching and sorting algorithms	
☐ Standard searching algorithms: ○ Binary search ○ Linear search ☐ Standard sorting algorithms: ○ Bubble sort ○ Merge sort ○ Insertion sort	**Required** ✓ Understand the main steps of the algorithm and the segments of code in it ✓ Understand any pre-requisites of an algorithm ✓ Apply the algorithm to a data set ✓ Identify an algorithm if given the code, pseudocode or Exam Reference Language for it **Not required** × To remember the code, pseudocode or Exam Reference Language for these algorithms



	Linear search (simple scan)	Binary search (split in half)
What it does	Checks items **one by one** until it finds the target (or reaches the end).	Repeatedly checks the **middle** and halves the search area.
Pre-requisite	**None** (list can be unsorted).	List must be **sorted** (usually ascending).
Main steps (in words)	1) Start at the first item. 2) Compare to target. 3) If match → **found**. 4) If not, move to next. 5) End reached → **not found**.	1) Set **low** and **high**. 2) Find **mid**. 3) Compare `list[mid]` to target: equal → found; bigger → right half (`low = mid + 1`); smaller → left half (`high = mid - 1`). 4) Repeat until found or `low > high`.
How to apply to a data set	Tick through each value in order. Count comparisons if asked.	Record **low / high / mid** each step (trace table). Cross off the half you throw away.
How to spot it in code / pseudocode	A loop that goes through the list from start to end (`FOR each item` / index +1). Often stops early if found.	Uses **low, high, mid**. Often `WHILE low <= high`. Updates `low = mid + 1` or `high = mid - 1`.
Mini example	Find 8 in `[3, 9, 1, 8, 5]` → 3 ❌, 9 ❌, 1 ❌, 8 ✅ found.	Find 23 in `[11, 15, 19, 23, 27]` → mid=19 → go right `[23, 27]` → mid=23 ✅ found.

BMI – Bubble Merge Insertion

Bubble Sort

Sort [5, 1, 4, 2] and show each pass.

Pass	Start of pass	Compare & swap steps (left → right)	End of pass
1	5 1 4 2	Compare 5 & 1 → swap Compare 5 & 4 → swap Compare 5 & 2 → swap	1 4 2 5
2	1 4 2 5	Compare 1 & 4 → no swap Compare 4 & 2 → swap Compare 4 & 5 → no swap	1 4 2 5 1 2 4 5
3	1 4 2 5	Compare 1 & 2 → no swap Compare 4 & 4 → no swap Compare 4 & 5 → no swap	1 2 4 5 1 2 4 5
3	1 2 4 5	Compare 1 & 2 → no swap Compare 2 & 4 → no swap Compare 4 & 5 → no swap	1 2 4 5 1 2 4 5 ✓

```
// Bubble Sort (simple pseudocode)
REPEAT
    swapped ← FALSE

    FOR i ← 0 TO length(list) - 2
        IF list[i] > list[i + 1] THEN
            temp ← list[i]
            list[i] ← list[i + 1]
            list[i + 1] ← temp
            swapped ← TRUE
        ENDIF
    NEXT i

UNTIL swapped = FALSE
```

Merge Sort

Sort [5, 1, 4, 2]

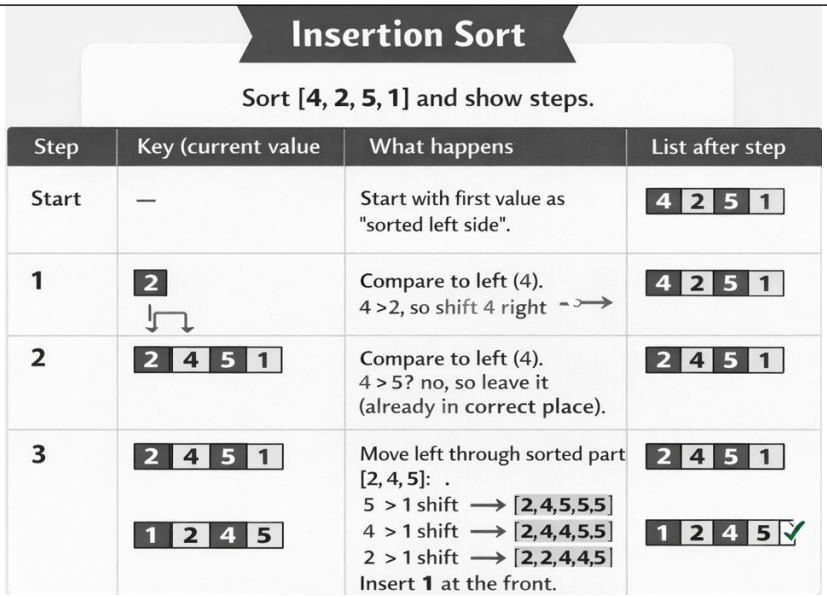
Stage	What you do	Result
Start	Original list	5 1 4 2
Split 1	Split into two halves	5 1 • → 4 2
Split 2	Split each half into single items	5 1 4 2
Merge (left half)	Merge [5] and [1] (take smallest first) ·Compare 5 vs 1 → take 1, then 5	1 5 1, 5
Merge (right half)	Merge [4] and [2] ·Compare 4 vs 2 → take 2, then 4	1 2 4 2, 4
Final merge	Merge [1,5] and [2,4] ·Compare 1 vs 2 → take 1 → [1] ·Compare 5 vs 2 → take 2 → [1, 2] ·Compare 5 vs 4 → take 4 → [1, 2, 4] ·Take remaining 5 →	1 1 2 4 5 1 2, 4 5 1 2, 4 5 5 ✓

```
FUNCTION Merge(left, right)
    result ← empty list

    WHILE left not empty AND right
                                not
                                empty
        IF first(left) <= first(right) THEN
            add first(left) to result
            remove first(left) from left
        ELSE
            add first(right) to result
            remove first(right) from
right
        ENDIF
    ENDWHILE

        add remaining items in left to
result
        add remaining items in right to
result

        RETURN result
    ENDFUNCTION
```



Insertion Sort

Sort [4, 2, 5, 1] and show steps.

Step	Key (current value	What happens	List after step
Start	—	Start with first value as "sorted left side".	4 2 5 1
1	2	Compare to left (4). 4 > 2, so shift 4 right →	4 2 5 1
2	2 4 5 1	Compare to left (4). 4 > 5? no, so leave it (already in correct place).	2 4 5 1
3	2 4 5 1 1 2 4 5	Move left through sorted part [2, 4, 5]: . 5 > 1 shift → [2,4,5,5] 4 > 1 shift → [2,4,4,5] 2 > 1 shift → [2,2,4,5] Insert **1** at the front.	2 4 5 1 1 2 4 5 ✓

```
// Insertion Sort pseudocode

FOR i ← 1 TO LENGTH(list) - 1
    key ← list[i]
    j ← i - 1

    WHILE j >= 0 AND list[j] > key
        list[j + 1] ← list[j]     // shift right
        j ← j - 1
    ENDWHILE

    list[j + 1] ← key          // insert key
NEXT i
```

	Bubble sort	Insertion sort	Merge sort
What it does	Compares **neighbours** and **swaps** if wrong order. Biggest values "bubble" to the end.	Builds a sorted section on the **left** by taking a **key** value and inserting it in the right place.	**Splits** the list into halves, then **merges** back in sorted order.
Main steps	Do a pass left→right: compare `i` and `i+1`, swap if needed. Repeat passes until no swaps	Assume first item is sorted. Take next item as **key**. Shift bigger items right. Insert key. Repeat.	Split list in half repeatedly until single items. Merge pairs by repeatedly taking the smallest front item.
Apply to data	Show each **pass** and the swaps. After each pass, the last item is in the correct place.	Show the "sorted left side" growing. Show **shifts** then where the key is placed.	Draw the **splits** (like a tree), then show the **merges** step-by-step.
Spot in code / pseudocode	Compares neighbours (`list[j]` vs `list[j+1]`). Often nested loops or a `swapped` flag.	Uses `key` (or current). Has a loop moving left while `list[j] > key`, shifting (`list[j+1] = list[j]`).	Mentions split into left/right, often recursion, and a clear **merge** step combining two sorted lists.
Mini example idea	`[5,1,4,2]` one pass → `[1,4,2,5]`(5 bubbled to end)	`[4,2,5,1]` → `[2,4]` → `[2,4,5]`→ `[1,2,4,5]`	Merge `[2,6]` and `[1,5]` → `[1,2,5,6]`

Bubble - Go along the list comparing **neighbours.** If two are in the wrong order, **swap** them. Keep doing passes until a full pass makes **no swaps** (then it's sorted).

Merge - Split the list in half again and again until you have single items. Then **merge** the pieces back together by always taking the **smallest front item** first. Merge pairs of **1**, then **2**, then **4**... items until the whole list is sorted.

Insertion - **works by splitting the list into a sorted and an unsorted part.**
One value at a time is taken from the unsorted list and inserted into the correct position in the sorted list by shifting larger values to the right until



a gap is created.

Merge sort – show every stage: [8,3,6,1,7,2,5,4] → split into [8,3,6,1] and [7,2,5,4] → then pairs → single values. Merge sorted pairs [3,8], [1,6], [2,7], [4,5]; merge halves [1,3,6,8], [2,4,5,7]; final [1,2,3,4,5,6,7,8]. Credit comes from showing the process, not just the final answer.

"The more we order the world, the more clearly it speaks to us."

— ***Sister Teresa / Pierre Teilhard de Chardin***

2.1.3 Exam Questions on Searching and Sorting Algorithms

Question	Tip to answer (student)
1. Which search algorithm **requires the data to be in order**? (Tick one) (1) ☐ Linear search ☐ Binary search Binary search [1]	Ordered list = binary.
2. State one condition that will stop a **linear search**. (1) The item is found **or** the end of the list is reached. [1]	Think: found OR end.
3. A student uses **binary search** on an unordered list. Explain why this can fail. (2) Binary search assumes the data is sorted.[1] If it isn't, the algorithm may discard the half containing the value.[2]	Mention "wrong half".
4. A list has 12 items. Explain why binary search is usually faster than linear search on large ordered lists. (3) Binary search checks the middle value first.[1] It throws away half the list each step.[2] So it needs fewer comparisons than checking every item one by one.[3]	Use "halves each time".
5. Complete the statement: "Linear search checks items _____ until the value is found." (1) one-by-one [1]	Simple word.
6. A bubble sort compares pairs of values and swaps them when they are... (Tick one) (1) ☐ in the correct order ☐ in the incorrect order ☐ always swapped in the incorrect order [1]	Bubble = swap wrong pairs.
7. A bubble sort makes a "pass" through a list. Describe what happens in one pass. (2) The algorithm compares each pair of neighbours.[1] It swaps them if they are in the wrong order.[2]	Compare + swap.
8. The data is: **[5, 1, 4, 2]**. Show the list **after one full pass** of bubble sort (ascending). (3) Compare 5 & 1 → swap → [1,5,4,2][1] Compare 5 & 4 → swap → [1,4,5,2][2] Compare 5 & 2 → swap → [1,4,2,5][3]	Swap neighbours left to right.
9. Explain why bubble sort may still do one extra pass even when the list looks sorted. (2) The algorithm can't "know" it is sorted.[1] It must complete a pass where **no swaps happen** to confirm it.[2]	Needs "no swaps" check.
10. An insertion sort works by keeping a _____ part and an _____ part of the list. (2) sorted[1] and unsorted[2]	OCR wording.
11. Describe one way insertion sort is different from bubble sort. (2)	Pick one clear difference.

Question	Tip to answer (student)

in the sorted part.[1] Bubble sort repeatedly swaps neighbouring items during passes.[2]

12. A merge sort uses "divide and conquer".
Describe what this means. (3)

Split then merge in order.

The list is split into smaller lists.[1] These lists are sorted by merging them back together in order.[2] The process continues until one sorted list remains.[3]

13. Show the **main steps** a merge sort would take to sort: **[7, 2, 9, 1]** into ascending order. (4)

Split → pairs → merge.

Split into single items: [7] [2] [9] [1][1] Merge into sorted pairs: [2,7] and [1,9][2] Merge into one sorted list: [1,2,7,9][3] Clear "divide then merge" steps shown.[4]

14. A searching algorithm checks the first item, then the next, and continues until it finds the value. Name the algorithm. (1)

First → next → next.

Linear search [1]

15. A binary search is being done on this ordered list: **[3, 8, 12, 19, 25, 31, 40]**. The value searched for is **25**.
State the middle value checked first and what happens next. (3)

Middle first, then half.

Middle value is 19.[1] 25 is bigger than 19 so search the right half.[2] New range becomes [25,31,40].[3]

16. A programmer says: "Binary search is always better than linear search."
Give one situation where **linear**search may be more suitable. (2)

Unsorted or tiny list.

If the list is not sorted, linear search works without sorting first.[1] For very small lists, linear search can be simpler and fast enough.[2]

17. **Complete the algorithm** for a linear search that outputs "FOUND" if value is in the list, otherwise "NOT FOUND". (6)

Fill the blanks only.

```
value = input()
found = FALSE
for i = 0 to length(list)-1
   if list[i] == _______ then
      found = _______
   endif
next i
if found == TRUE then
   print("_________")
else
   print("_____________")
endif
```

value[1] TRUE[2] FOUND[3] NOT FOUND[4] Correct comparisons and outputs used.[5] Uses found flag to decide final output.[6]

2.2 Programming Fundamentals

02 KEY VOCABULARY / 2.1.3

Searching and sorting

Use the correct technical wording before attempting longer answers.

Linear search

Examines items one after another; the data need not be sorted.

Binary search

Searches a sorted list by repeatedly excluding half the remaining items.

Bubble sort

Compares neighbouring items and swaps them when they are out of order.

Insertion sort

Builds an ordered part by inserting each next value in position.

Merge sort

Divides a list and merges smaller sorted sublists.

03 WORKED EXAMPLE / 2.1.3

Searching and sorting

Follow a complete example. Notice the method before attempting the separate questions.

Task
Find 10 using binary search in a sorted list.

Think it through — small steps

1. Start with [2, 4, 6, 8, 10, 12, 14]. The list is already sorted.
2. Compare the target 10 with the middle item, 8: search the right half.
3. The next middle is 12: search the left part of that half.
4. The remaining item is 10: found in three comparisons.

Why this earns marks
The precondition (sorted data) makes it valid to discard half the list each time.

SPEC 2.1.3 | SKILL 05 OF 28

Linear search

REMEMBER

Linear search checks items one by one and can search an unsorted list.

THINK OF IT LIKE...

Looking through a pile of cards from top to bottom.

WORKED EXAMPLE / APPLY

Find 9 in [3, 9, 1]: check 3, then 9; stop at position 2.

YOU DO - EXAM PRACTICE

Explain why linear search can be used on an unsorted list. (2)

Write your answer before opening the separate answer section.

SPEC 2.1.3 | SKILL 06 OF 28

Binary search

REMEMBER

Binary search requires sorted data and repeatedly eliminates half the remaining search range.

THINK OF IT LIKE...

Finding a word in a sorted dictionary.

WORKED EXAMPLE / APPLY

Find 8 in [2, 4, 6, 8, 10, 12, 14]: compare middle 8 and stop.

YOU DO - EXAM PRACTICE

State a precondition of binary search and explain why it matters. (2)

Write your answer before opening the separate answer section.

SPEC 2.1.3 | SKILL 07 OF 28

Bubble sort

REMEMBER

Bubble sort compares adjacent items and swaps those in the wrong order until no swaps are needed.

THINK OF IT LIKE...

Neighbouring cards change places until ordered.

WORKED EXAMPLE / APPLY

One ascending pass over [5, 2, 4] gives [2, 4, 5].

YOU DO - EXAM PRACTICE

Give the list after one full ascending bubble-sort pass on [4, 1, 3]. (2)

Write your answer before opening the separate answer section.

SPEC 2.1.3 | SKILL 08 OF 28

Insertion sort

REMEMBER

Insertion sort builds a sorted section by inserting each new item in its correct place.

THINK OF IT LIKE...

Insert a playing card into a sorted hand.

WORKED EXAMPLE / APPLY

Sorted part [2, 6]; next 4 -> [2, 4, 6].

YOU DO - EXAM PRACTICE

Explain how insertion sort places a new value into a partially sorted list. (2)

Write your answer before opening the separate answer section.

SPEC 2.1.3 | SKILL 09 OF 28

Merge sort

REMEMBER

Merge sort splits a list into small sublists and combines them in sorted order.

THINK OF IT LIKE...

Sorting two already ordered stacks of cards into one.

WORKED EXAMPLE / APPLY

Merge [1, 5] with [2, 4] -> [1, 2, 4, 5].

YOU DO - EXAM PRACTICE

Describe the divide and merge stages of merge sort. (2)

Write your answer before opening the separate answer section.

04 INDEPENDENT EXAM PRACTICE / 2.1.3

Searching and sorting

Do this without viewing the next answer pages. Use a whiteboard first for code/traces.

EXAM-STYLE CHALLENGE · 6 MARKS

Sort [6, 1, 5, 2] into ascending order using bubble sort. Show the list after the FIRST full pass and after the SECOND full pass. Explain why the algorithm must compare adjacent values, and state when it can stop.

Your response

PREDICT → WRITE → TEST → IMPROVE | Self-mark after finishing the entire topic.

MARK SCHEME | SKILL 05 OF 28

Linear search

QUESTION

Explain why linear search can be used on an unsorted list. (2)

MODEL ANSWER

It compares the target with each item sequentially (1). It does not depend on items being in any particular order (1).

SELF-MARK

Give credit for equivalent correct wording. Correct your answer in a different colour, then revisit the study card if a key point was missing.

MARK SCHEME | SKILL 06 OF 28

Binary search

QUESTION

State a precondition of binary search and explain why it matters. (2)

MODEL ANSWER

The list must be sorted (1). Comparing with the middle item then indicates which half can safely be discarded (1).

SELF-MARK

Give credit for equivalent correct wording. Correct your answer in a different colour, then revisit the study card if a key point was missing.

MARK SCHEME | SKILL 07 OF 28

Bubble sort

QUESTION

Give the list after one full ascending bubble-sort pass on [4, 1, 3]. (2)

MODEL ANSWER

Compare 4 and 1, swap to [1, 4, 3] (1). Compare 4 and 3, swap to [1, 3, 4] (1).

SELF-MARK

Give credit for equivalent correct wording. Correct your answer in a different colour, then revisit the study card if a key point was missing.

MARK SCHEME | SKILL 08 OF 28

Insertion sort

QUESTION

Explain how insertion sort places a new value into a partially sorted list. (2)

MODEL ANSWER

It compares the new item with items in the sorted portion and shifts larger items to the right (1). It inserts the new item into the resulting gap (1).

SELF-MARK

Give credit for equivalent correct wording. Correct your answer in a different colour, then revisit the study card if a key point was missing.

MARK SCHEME | SKILL 09 OF 28

Merge sort

QUESTION

Describe the divide and merge stages of merge sort. (2)

MODEL ANSWER

Divide the list into progressively smaller sublists (1). Merge sublists by repeatedly taking the next smallest value (1).

SELF-MARK

Give credit for equivalent correct wording. Correct your answer in a different colour, then revisit the study card if a key point was missing.

05 ANSWERS & MARK SCHEME / 2.1.3

Searching and sorting

This answer belongs to the exam-style challenge on the previous practice page.

6-mark challenge · Model answer
Bubble-sort passes

Mark 1 First comparison: 6 and 1 swap; next compare 6 and 5.

Mark 2 After first complete pass: [1, 5, 2, 6].

Mark 3 Second pass swaps 5 and 2.

Mark 4 After second complete pass: [1, 2, 5, 6].

Mark 5 Bubble sort compares adjacent items and swaps any pair in the wrong ascending order.

Mark 6 It can stop after a full pass produces no swaps.

Accept equivalent correct methods, code or relevant wording. Award each point once.